Язык программирования C++

Введение в новые стандарты

Доцент кафедры ИВТ, к.т.н.
Проскурин Александр Викторович

Стандарты языка C++

C++98	C++11	C++14	C++17	C++20
1998	2011	2014	2017	2020
• Шаблоны • Библиотека STL • Строки (string) • Потоки ввода/вывода (i/o stream)	• Семантика перемещения (move) • Обобщённые константные выражения (conexpr) • Вывод типов (auto) • for-цикл по коллекции • Универсальная инициализация • std::array • Лямбда-функции • Умные указатели • Регулярные выражения	• Механизм блокировки чтения/записи • Обобщённые лямбда-функции	• Параллельные алгоритмы STL • Библиотека файловой системы • Типа данных: std::optional, std::variant и std::any	• Библиотека диапазонов (ranges) • Сопрограммы (coroutines) • Модули (modules) • Концепции (concepts)

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Uniform initialization

В C++11 инициализация переменных получила единообразный синтаксис.

Например:

```
// Инициализация до C++11
int a = 5;
float b[] = { 2.3f, 1.9f };
vector<int> v;
for (int i = 0; i < 4; i++)
    v.push_back(i);

// Инициализация в C++11
int a{ 5 };
float b[]{ 2.3f, 1.9f };
vector<int> v{ 1, 2, 3, 4 };
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Инициализаторы нестатических членов-данных

Нестатические поля классов можно инициализировать там, где они объявлены, а не только в конструкторах.

Например:

```
// Инициализация по умолчанию до C++11
class Human {
private:
    unsigned age;
public:
    Human() : age(0) {}
};
// Инициализация по умолчанию в C++11
class Human {
private:
    unsigned age{ 0 };
};
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Константа нулевого указателя

В C++ NULL – это константа 0, что может привести к нежелательному результату при перегрузке функций:

```
void f (char *);
void f (int);
```

При обращении f(NULL) будет вызвана f (**int**); ,что не совпадает с планами программиста.

В C++11 введено новое ключевое слово **nullptr** для описания константы нулевого указателя:

```
std::nullptr_t  nullptr;
```

где тип nullptr_t можно неявно конвертировать в тип любого указателя и сравнивать с любым указателем.

Неявная конверсия в целочисленный тип **недопустима**, за исключением **bool** (в целях совместимости).

Для обратной совместимости константа 0 также может использоваться в качестве нулевого указателя.

```
bool b = nullptr;  // OK: b = false;
int i = nullptr;   // Err!
f(nullptr);  // вызывается f(char*) а не f(int).
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

rvalue-ссылки

В C++11 появился новый тип данных – rvalue-ссылка:

```
тип && имя = временный_объект;
```

Можно использовать перегруженные функции для неконстантных временных объектов, обозначаемых посредством rvalue-ссылок.

Например:

```
class A;              A a;
void f (A & x);       f (a);
void f (A && y);      f ( A() );
…
A&& rr1 = A();
A&& rr2 = a; // Ошибка!
```

Автоматическое определение типа со значениями lvalue и rvalue:

```
int x = 0; // x – lvalue типа int
int& xl = x; // xl – lvalue типа int&
int&& xr = x; // Ошибка компиляции: x – lvalue
int&& xr2 = 1 + 2; // xr2 – lvalue типа int&&
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Семантика переноса (move semantics)

При создании/уничтожении временных объектов **неплоских классов** (содержащих указатели на динамическую память) требуется выделение/освобождение динамической памяти, что может отнимать много времени.

Можно оптимизировать работу с временными объектами неплоских классов, если не освобождать их динамическую память, а просто **перенаправить указатель на неё в объект, который копирует значение временного объекта неплоского класса** (посредством поверхностного копирования). При этом после копирования надо обнулить соответствующие указатели у временного объекта, чтобы его деструктор ее не зачистил.

Это возможно сделать с помощью перегруженных конструктора копирования и операции присваивания с параметрами – **rvalue-ссылками**. Их называют конструктором переноса (**move constructor**) и операцией переноса (**move assignment operator**).

При этом компилятор сам выбирает нужный метод класса, если его параметром является временный объект.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Семантика переноса. Пример

```
class Str {
    char* s{ nullptr };
    int len{ 0 };
public:
    Str(const char* sss = nullptr); // обычный конструктор класса
    Str(const Str& x); // традиционный конструктор копирования
    Str(Str&& x) noexcept {  // move constructor
        swap(x); // вызов метода swap
    }
    Str& operator = (const Str& x); // обычная перегруженная операция =
    Str& operator = (Str&& x) noexcept { // move assignment operator
        swap(x); // вызов метода swap
        return *this;
    }
    ~Str() { delete[] s; } // традиционный деструктор неплоского класса
    void swap(Str& x) noexcept {
        std::swap(s, x.s);
        std::swap(len, x.len);
    }
    Str operator + (const Str& x);
};
```

`noexcept` – метод не выбрасывает исключений

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Семантика переноса. Пример

Использование rvalue-ссылок в описании методов класса Str приведет к более эффективной работе, например, следующих фрагментов программы:

Пример 1:

```
Str a("abc"), b("def"), c;
c = b + a;  //  Str& operator= (Str &&);
```

Пример 2:

```
Str f(Str a) {
    ...
    return a;
}

Str d = f(Str("dd"));  // Str (Str &&);
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Обобщенные константные выражения (constexpr)

Введено ключевое слово **constexpr**, которое позволяет компилятору вычислить его ещё на этапе компиляции и использовать как константу.

Пример 1:

```
constexpr int square(int x) {
    return x * x;
}

// создание массива из 4 элементов
constexpr int a = square(2);
float b[a];
```

Пример 2:

```
constexpr double a = 9.8;
constexpr double b = a / 6;
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Вывод типов (auto)

Описание **явно инициализируемой переменной** может содержать ключевое слово **auto** – при этом типом созданной переменной будет тип инициализирующего выражения.

Пример:

```
auto a = 3.14; // double
auto b = 1; // int
auto& c = b; // int&
auto d = { 0 }; // std::initializer_list<int>
auto g = new auto(123); // int*
const auto h = 1; // const int
auto i = 1, j = 2, k = 3; // int, int, int
auto l = 1, m = true, n = 1.61; // ошибка – " l " выводится как
                                // int, " m " является bool
auto o; // ошибка – `o` требует инициализатора
```

Ещё пример:

Пусть ft(....) – шаблонная функция, которая возвращает значение шаблонного типа, тогда при описании:

```
auto var1 = ft(....); // var1 имеет соответствующий шаблонный тип
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Вывод типов. Продолжение

Это существенно улучшает читаемость:

```
std::vector<int> v = ...;
std::vector<int>::const_iterator cit = v.cbegin();
// против
auto cit = v.cbegin();
```

Функции также могут выводить возвращаемый тип с помощью auto:

```
template <typename X, typename Y>
auto add(X x, Y y) -> decltype(x + y) {
    return x + y;
}

add(1, 2); // == 3
add(1, 2.0); // == 3.0
add(1.5, 1.5); // == 3.0
```

`decltype` – оператор, возвращающий объявленный тип переданного ему выражения. Квалификаторы (const) и ссылки сохраняются, если они являются частью выражения.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

For-цикл по коллекции

Введена новая форма цикла for (**foreach**), позволяющая автоматически осуществлять **перебор элементов коллекции** (статические массивы и любые другие коллекции, для которых определены функции begin() и end()).

Синтаксис цикла следующий:

```
for (тип имя_элемента : массив_контейнер) {
   Блок действий;
}
```

Выполняется итерация по каждому элементу массив_контейнер, присваивая значение текущего элемента переменной имя_элемента.

В целях улучшения производительности объявляемый элемент должен быть того же типа, что и элементы массива, иначе произойдет неявное преобразование. Это идеальный случай для использования ключевого слова **auto**.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

For-цикл по коллекции. Пример

```
int arr[]{ 1, 2, 3, 4, 5 };
for (auto elem : arr) {  // итерации по массиву arr
    cout << elem << " "; // получаем доступ к элементу
}                        // массива через переменную x
```

При выполнении цикла foreach переменной **elem** присваивается значение первого элемента (т.е. значение 1).

Дальше программа выполняет блок действия по выводу значения переменной **elem**, т.е. 1.

Затем цикл for выполняется снова, и значением переменной **elem** уже является 2 (второй элемент массива).

Вывод значения **elem** выполняется снова.

Цикл продолжает свое выполнение до тех пор, пока в массиве не останется непройденных элементов.

При этом из-за использования auto **в переменную elem помещаются копии значений**.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

For-цикл по коллекции. Ссылки

Для того, чтобы работать с самим элементом, а не его копией, достаточно добавить ссылку:

```
vector<int> vec{ 10, 8, 6, 5, 4 };
for (auto& elem : vec) { // итерации по вектору vec
    elem *= 2; // увеличиваем значение элементов вдвое
}
```

При указании обычной ссылки, любые изменения элемента будут влиять на сам массив, что не всегда может быть желательно.

В связи с этим иногда (в зависимости от задачи) целесообразно сделать объявляемый элемент константным, тогда его можно использовать в режиме «только для чтения»:

```
/* elem – это константная ссылка на текущий элемент вектора в итерации */
for (const auto &elem : vec)
    std::cout << elem << ' ';
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Лямбда-функции

В стандартном C++, например, при использовании алгоритмов стандартной библиотеки C++ **sort**, **find** или **for_each**, часто возникает потребность в определении функций-предикатов рядом с местом, где осуществляется вызов этого алгоритма.

```
class PrintFunctor {
    void operator()(int x) const {
        std::cout << x << std::endl;
    }
};
int main() {
    std::vector<int> v;
    v.push_back(1);
    v.push_back(2);
    std::for_each(v.begin(), v.end(), PrintFunctor());
}
```

Зачастую данный способ является слишком избыточным и многословным и лишь затрудняет чтение кода.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Лямбда-функции. Продолжение

Для решения этой проблемы в C++11 предложены лямбда-функции. Это безымянные функции, способные захватывать переменные в области видимости. У них есть список захвата:

`[]` - ничего не захватывает.
`[ = ]` - захват всех локальных объектов по значению.
`[ & ]` - захват всех локальных объектов по ссылке.
`[this]` - захват указателя `this` по значению.
`[a, &b]` - захват объекта `a` по значению, `b` – по ссылке.

Тип возвращаемого значения определяется исходя из типа возвращаемого значения, что ограничивает размер лямбда-функции до одного выражения.

```
std::for_each(v.begin(), v.end(), [](int x) {
    std::cout << x << '\n'; });

int x = 1;
auto addX = [=](int y) { return x + y; };
addX(1); // == 2
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Улучшение конструкторов объектов

В отличие от старого стандарта новый стандарт C++11 позволяет вызывать одни конструкторы класса (так называемые **делегирующие конструкторы**) из других, что в целом позволяет избежать дублирования кода.

Пример:

```
class A {
    int* x{ nullptr };
    int n{ 0 };
public:
    A(int _n) : n(_n), x(new int[_n]) {}
    A(int _n, int _v) : A(_n) {
        // Память уже была выделена,
        for (auto& i : std::span(x, n))
            i = _v; // поэтому просто заносим значения
    }
};
```

`std::span(x, n)` – начиная с C++20 позволяет перебирать в цикле for элементы динамического массива.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Явные методы преобразования

Методы преобразования теперь можно сделать явными с помощью спецификатора explicit.

Пример:

```
class A {
    int n{ 10 };
public:
    explicit A(int x) : n(x) { }
    A() { }
};

A obj{ 3 }; // Ок
A b = 5; // Ошибка. Нет вызова конструктора A(int x)
```

Замечание: Если до конца проработал хотя бы один делегирующий конструктор, то его объект уже считается **полностью созданным**.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Явное замещение виртуальных методов и финальность

В C++11 добавлена возможность (с помощью спецификатора **override**) отследить ситуации, когда виртуальная функция в базовом классе и в производных классах имеет разные прототипы, например, в результате случайной ошибки (что приводит к тому, что механизм виртуальности для такой функции работать не будет).

Кроме того, введен спецификатор **final**, который обозначает следующее:

- в описании классов – то, что они не могут быть базовыми для новых классов;
- в описании виртуальных методов – то, что возможные производные классы от рассматриваемого не могут иметь виртуальные методы, которые бы замещали финальные методы.

Замечание: спецификаторы **override и final** имеют специальные значения только в приведенных на следующем слайде ситуациях.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Явное замещение виртуальных методов и финальность. Продолжение

```
class B {
    virtual void  some_func();
    virtual void  f(int);
    virtual void  g() const;
};

class D1 : public B {
    void  sone_func()  override;  // Err!: нет такой в B
    void  f(int) override;        // OK!
    void  f(long) override;   // Err!: несоответствие типа
    // Err: несоответствие квалификации функции
    void  f(int) const  override; // Err!
    // Err: несоответствие типа результата
    int   f(int) override;         // Err!
    void  g() const final;         // OK!
    virtual void  g(long);  // OK: новая виртуальная функция
};
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Явное замещение виртуальных методов и финальность. Продолжение

Продолжение примера:

```
class D2 : D1 { // см. предыдущий слайд
    // Err: замещение финальной функции
    virtual void g() const; // Err!
};

class F final {
    int x, y;
};

// Err: наследование от  финального класса
class D : F { // Err!
    int z;
};
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Перечисления со строгой типизацией

В C++ :

- перечислимый тип данных фактически совпадает с целым типом,
- если перечисления заданы в одной области видимости, то имена их констант не могут совпадать.

В C++11 предложено использовать объявление **enum class** (или **enum struct**), чтобы избежать этих недостатков.

Например:

```
enum class  E {
   V1,  V2,  V3 = 100,  V4 /*101*/
};
```

Элементы такого перечисления нельзя неявно преобразовать в целые числа:

```
E::V4 == 101; // Err! Ошибка компиляции
```

Однако теперь перечисление является отдельным типом, что затрудняет чтение и вывод в поток.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

sizeof для членов данных классов без создания объектов

В C++11 разрешено применять операцию sizeof к членам-данным классов независимо от объектов классов.

Пример:

```
class A
{
  some_type a;
};
...
sizeof(A::a) // OK!
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Кортежи и привязки

Появились в C++11. Представляют собой коллекции с фиксированной длиной. Кортежи можно использовать, чтобы вернуть несколько значений из функции:

```
#include <tuple>

auto get() {
    return std::make_tuple("fred"s, 42);
}
```

s в конце литерала строки создаёт строку типа string, а не const char*.

Для получения отдельных значений можно использовать три способа:

1) С помощью std::get

```
auto t = get();
std::cout << std::get<0>(t) << std::endl;
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Кортежи и привязки. Продолжение

```
auto get() {
    return std::make_tuple("fred"s, 42);
}
```

Для получения отдельных значений можно использовать три способа:

2) С помощью std::tie

```
std::string name;
int age;
std::tie(name, age) = get();
std::cout << name << " is " << age << std::endl;
```

3) С помощью auto

```
auto [name, age] = get();
std::cout << name << " is " << age << std::endl;
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева